

Variabili,Costanti,Comments Code

Dalla versione ECMAScript 6 la dichiarazione di una variabile avviene attraverso la keyword 'let', in una costante con la keyword 'const'.

Se è la prima volta che studiate programmazione vi starete chiedendo: Cosa si intende per variabile, keyword e costante?

La keyword in un linguaggio di programmazione può essere paragonato ad un 'id' fisso di riconoscimento del dato.

Se scrivo in javascript `let numero = 3;` il linguaggio javascript capisce che sto assegnando una variabile passando un valore di tipo 'Number'.

Una variabile viene detta dichiarata se è caratterizzata soltanto da un nome:

`let numero;` (undefined) dichiarata ed inizializzata se le si dà anche un valore: `let numero = 5;` l' '=' è l'operatore in cui viene assegnato un valore alla variabile e il ';' è il segno di fine istruzione per poter dichiarare e inizializzare una variabile correttamente. Una variabile una volta dichiarata ed inizializzata verrà memorizzata e allocata nel nostro file.js come un contenitore fino a quando la utilizzeremo.

Una costante viene dichiarata ed inizializzata allo stesso modo di una variabile con l'unica differenza che non è possibile ridefinirla in seguito.

Esempio

variabile	costante	
<code>let nome = 5;</code> <code>nome = 6;</code>	<code>const nome = 5;</code>	<code>const nome = 5;</code> <code>nome = 6;</code>
Va bene	Va bene	Non va bene

Esempio di short program in javascript let

```
numero = 5;
Console.log(numero); Output
Console: 5
```

È possibile utilizzare il `console.log` come output del programma utilizzando la console debug nel browser Chrome oppure se state già lavorando con HTML è possibile utilizzare il `document.getElementById` visualizzando l'output nel body della pagina web, oppure il metodo `document.write` ma è consigliato solo in fase di test.

Nell'esempio precedente ho voluto darvi un'idea di un semplice algoritmo tra variabile inizializzata e come ottenere l'output della variabile in JavaScript mediante il `console.log`.

Altra nozione molto importante per le variabili riguarda variabile per valore e per riferimento.

Esempio

valore

riferimento

`let numero = 3;`

`let num = numero;`

Qual è la differenza?

Nel primo caso alla variabile viene passato un valore uguale a 3.

Nel secondo caso dichiaro una seconda variabile non passandole un valore ma un riferimento ovvero il nome del dato precedentemente dichiarato.

I commenti in JavaScript servono a far ignorare al nostro interprete (browser)

il codice vecchio ad esempio compilato in precedenza e può avvenire in due modi: singola riga – su più righe di codice.

Esempio

singola riga

```
// let num= 3;
```

più righe

```
/*let num = 2;
```

```
let n = 15;*
```

Operatori aritmetici

Gli operatori aritmetici utilizzati in javascript sono i seguenti:

+ esegue un addizione o concatenazione tra stringhe

- esegue una sottrazione tra valori numerici

/ esegue una divisione

* esegue il prodotto tra valori numerici

% operatore modulo calcola il resto di una divisione

++operatore di incremento di iterazione

-- operatore di decremento di iterazione

Operatori relazionali o di confronto

= assegnamento

== uguale a

=== identico a

<= minore uguale

>= maggiore uguale

!= non uguale a

Operatori logici

true o false = vero o falso

&& AND restituisce true se entrambi le espressioni risultano vere altrimenti restituisce false

Esempio: (5>3&&1<2) true

|| OR restituisce false se entrambe le espressioni restituiscono false altrimenti restituisce true

Esempio (5<3&&1>2)false

! negazione = cambia il valore di un espressione.

Operatore Ternario

```
let num = 5<3?true:false; Console.log(num);
```

Ho assegnato alla variabile num l'espressione da verificare, ottenendo in output false o true in base alla verifica del compilatore.

? restituirà true in caso 5<3 in caso contrario :(altrimenti) restituirà false.

Console.log vs Document.write vs Document.getElementById

In javascript si utilizzano più metodi per visualizzare l'output di un programma, document.write è consigliabile utilizzarlo solo in fase di test del codice in quanto andrebbe a cancellare l'html presente della pagina web,

il console.log è possibile utilizzarlo in fase di debug o ispezione del codice nella console del browser,

il document.getElementById viene utilizzato quando si gestisce il DOM con javascript e l'output verrà visualizzato nel body della pagina web del browser.

Costrutti di controllo

Il flusso control code in javascript può avvenire con l'operatore ternario ?: oppure attraverso i comandi if else else if.

If-else-else if

```
If (5<3){  
  Console.log(false);  
  
} else if(5>3){  
  Console.log(true);  
  
}
```

If (se) 5<3 espressione da valutare { blocchi di codice da inserire} nel nostro caso Console.log per ottenere l'output boolean, else if (altrimenti se) la prima verifica non viene soddisfatta ne viene verificata una successiva.

Switch

```
let mese = 3;  
  
Switch(mese){ case  
1:  
  console.log('marzo');
```

break; (interrompe) case 2:

```
console.log('marzo'); break;
```

default:

```
console.log('aprile');
```

```
}
```

Il comando Switch valuta l'espressione, confronta il valore dell'espressione con le clausole case , qualora non trovasse il valore

aspettato (1,2) utilizza l'operatore default.

In questo esempio in output otterremo il valore 'aprile'.

Costrutti di iterazione

Do-while

```
let c = 1; do{
```

```
console.log(c); c+=3;
```

```
}while (c<10);
```

Il ciclo do-while come da codice di esempio esegue almeno un iterazione anche se risulta falsa.

While

```
let b= 0;

while (b<18){ b+=2;

console.log(b);

}
```

Il ciclo While a differenza del Do-While itera finchè una condizione risulta vera.

For

```
let a;

for (a=0;a<10;a++){ If

(a==4){

continue;

}

    console.log(a);

}
```

Il ciclo for è caratterizzato da inizializzazione-condizione-iterazione, nel codice di esempio a viene incrementa di un valore fino a che sarà minore di 10. Il comando continue indica al ciclo for di ignorare il valore uguale a 4 nell'iterazione.